

CONFIGURING HTTPS ON APACHE SERVER IN KALI LINUX:

Report By- Mubasshera Zaidy

INTRODUCTION

Apache server → Apache HTTP Server is one of the most widely used web servers. It serves websites and web applications.

• **HTTPS** → HyperText Transfer Protocol Secure. It's just HTTP but encrypted using SSL/TLS certificates. This ensures:

- Data between client (browser) and server is encrypted.
- Prevents eavesdropping and MITM attacks.
- Provides authentication (the browser knows it's talking to the real server).

• **Configure HTTPS** → Setting up Apache so that it can:

- Use port **443** (default for HTTPS).
- Load an SSL/TLS certificate (self-signed or CA-issued).
- Encrypt traffic using that certificate.

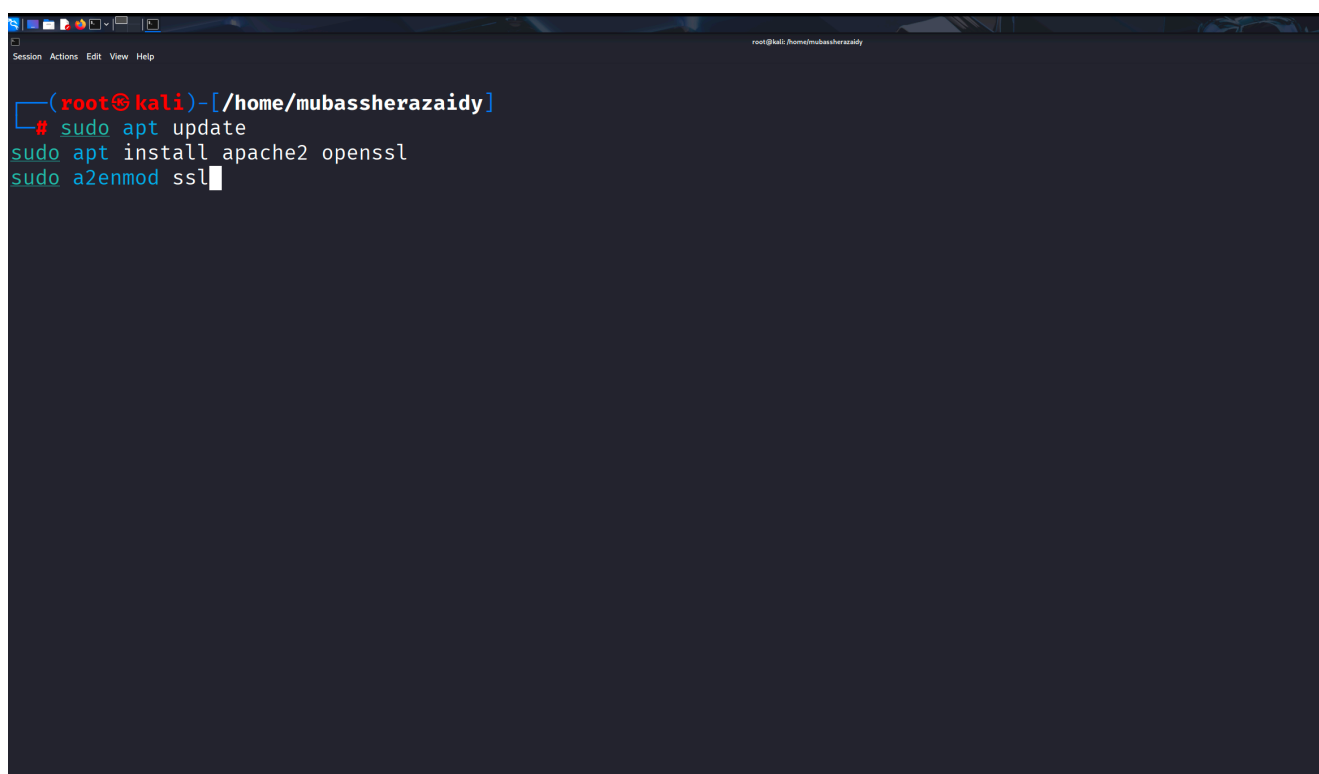
OBJECTIVE

To Configure HTTPS on Apache server on Kali Linux.

METHODOLOGY

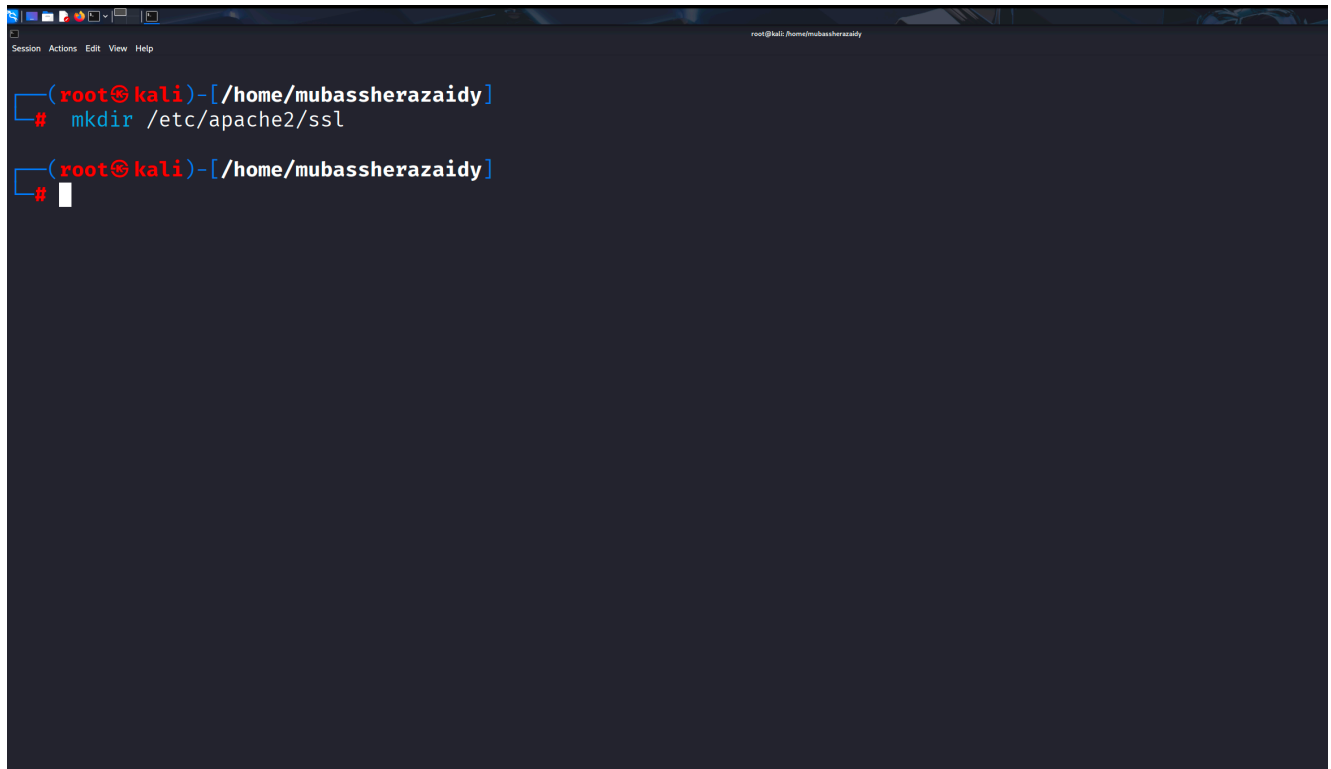
STEP 1 : In your kali terminal run the following commands to install Apache and SSL module:

```
sudo apt update
sudo apt install apache2 openssl
sudo a2enmod ssl
```

A screenshot of a Kali Linux terminal window. The window has a dark background and a light-colored title bar. The terminal prompt is `(root@kali)-[/home/mubassherazaidy]`. The user has entered three commands: `# sudo apt update`, `sudo apt install apache2 openssl`, and `sudo a2enmod ssl`. The cursor is at the end of the third command.

```
(root@kali)-[/home/mubassherazaidy]
# sudo apt update
sudo apt install apache2 openssl
sudo a2enmod ssl
```

STEP 2 : Create the SSL directory using `mkdir /etc/apache2/ssl` command.

A terminal window with a dark background. The title bar shows 'root@kali: /home/mubassherazaidy'. The prompt is '(root@kali)-[/home/mubassherazaidy]'. The command 'mkdir /etc/apache2/ssl' has been entered and executed. The prompt is now '(root@kali)-[/home/mubassherazaidy]#' with a cursor at the end.

```
(root@kali)-[/home/mubassherazaidy]
# mkdir /etc/apache2/ssl
(root@kali)-[/home/mubassherazaidy]
#
```

STEP 3 : Use the following command to generate a self-signed certificate and a private key:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/apache2/ssl/apache.key \
-out /etc/apache2/ssl/apache.crt
```

- req: Use the “certificate request” tool in OpenSSL.
- -x509: Output an X.509 certificate (self-signed) rather than a CSR. This means OpenSSL signs the certificate with the key you’re generating, so no external CA is needed.
- -nodes: “No DES.” Do not encrypt the private key with a passphrase. Apache can read the key at startup without asking you for a password.
- -days 365: Set certificate validity to 365 days. After that, you must renew/regenerate.
- -newkey rsa:2048: It will generate a new RSA private key of 2048 bits.
- -keyout /etc/apache2/ssl/apache.key: This command will write the private key to this file path.
- -out /etc/apache2/ssl/apache.crt: This command will write the resulting certificate to this file path.

[illegible]

STEP 4 : Enter the required fields with appropriate entries.

[illegible]

STEP 5 : Run the following command to enable the default SSL site : `sudo a2ensite default-ssl.conf`

[illegible]

STEP 6 : Use the following command to open and edit Apache's configuration file in the terminal : `nano /etc/apache2/sites-available/default-ssl.conf`

[illegible]

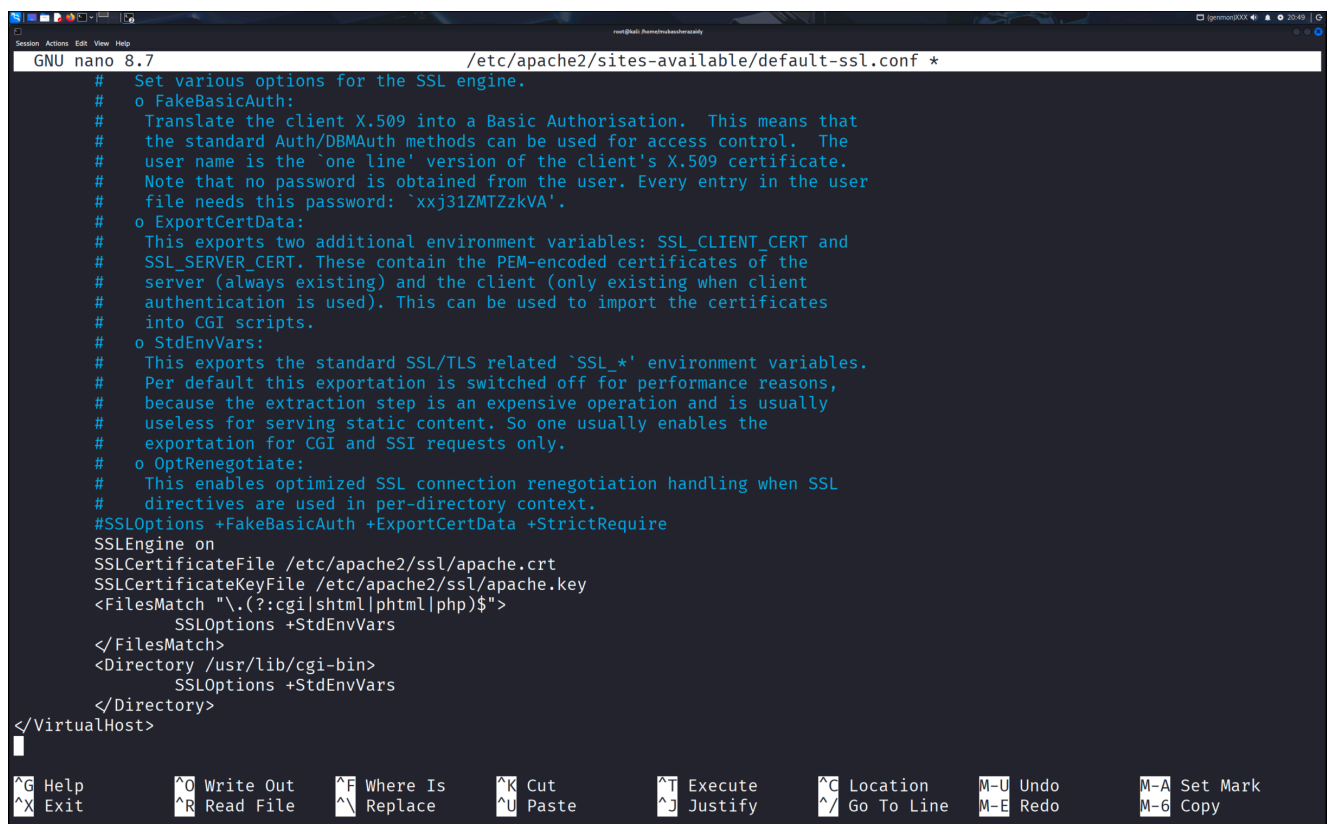
STEP 7 : Inside the file, type :

SSL Engine on

SSLCertificateFile /etc/apache2/ssl/apache.crt

SSLCertificateKeyFile /etc/apache2/ssl/
apache.key

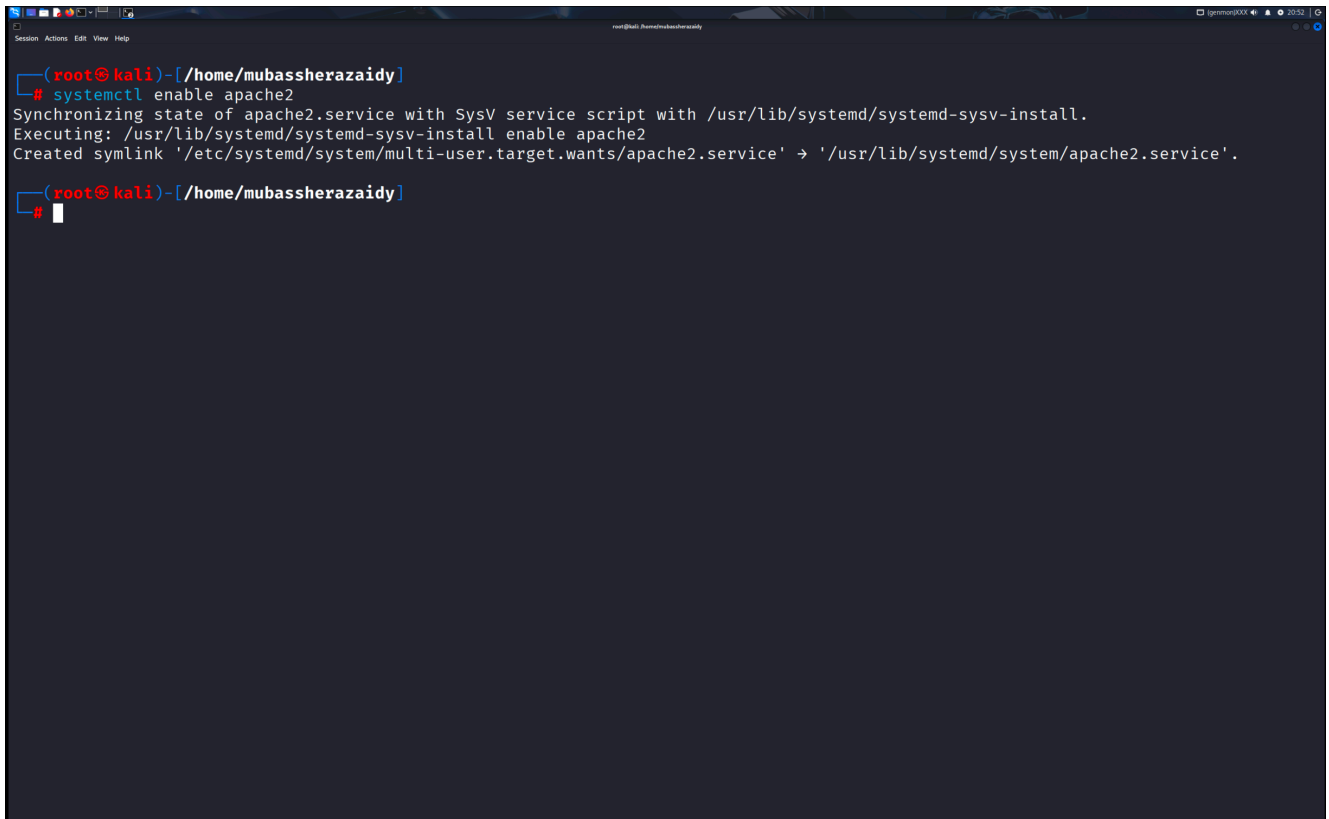
Without editing the file, Apache won't know where to find the
SSL files.



```
GNU nano 8.7 /etc/apache2/sites-available/default-ssl.conf *
# Set various options for the SSL engine.
#
# o FakeBasicAuth:
#   Translate the client X.509 into a Basic Authorisation. This means that
#   the standard Auth/DBMAuth methods can be used for access control. The
#   user name is the 'one line' version of the client's X.509 certificate.
#   Note that no password is obtained from the user. Every entry in the user
#   file needs this password: 'xxj31ZMTZzkVA'.
#
# o ExportCertData:
#   This exports two additional environment variables: SSL_CLIENT_CERT and
#   SSL_SERVER_CERT. These contain the PEM-encoded certificates of the
#   server (always existing) and the client (only existing when client
#   authentication is used). This can be used to import the certificates
#   into CGI scripts.
#
# o StdEnvVars:
#   This exports the standard SSL/TLS related 'SSL_*' environment variables.
#   Per default this exportation is switched off for performance reasons,
#   because the extraction step is an expensive operation and is usually
#   useless for serving static content. So one usually enables the
#   exportation for CGI and SSI requests only.
#
# o OptRenegotiate:
#   This enables optimized SSL connection renegotiation handling when SSL
#   directives are used in per-directory context.
#SSLOptions +FakeBasicAuth +ExportCertData +StrictRequire
SSL Engine on
SSLCertificateFile /etc/apache2/ssl/apache.crt
SSLCertificateKeyFile /etc/apache2/ssl/apache.key
<FilesMatch "\.(?:cgi|shtml|phtml|php)$">
    SSLOptions +StdEnvVars
</FilesMatch>
<Directory /usr/lib/cgi-bin>
    SSLOptions +StdEnvVars
</Directory>
</VirtualHost>
```

Help **Write Out** **Where Is** **Cut** **Execute** **Location** **M-U** **Undo** **M-A** **Set Mark**
X **Exit** **R** **Read File** **** **Replace** **U** **Paste** **J** **Justify** **/** **Go To Line** **M-E** **Redo** **M-6** **Copy**

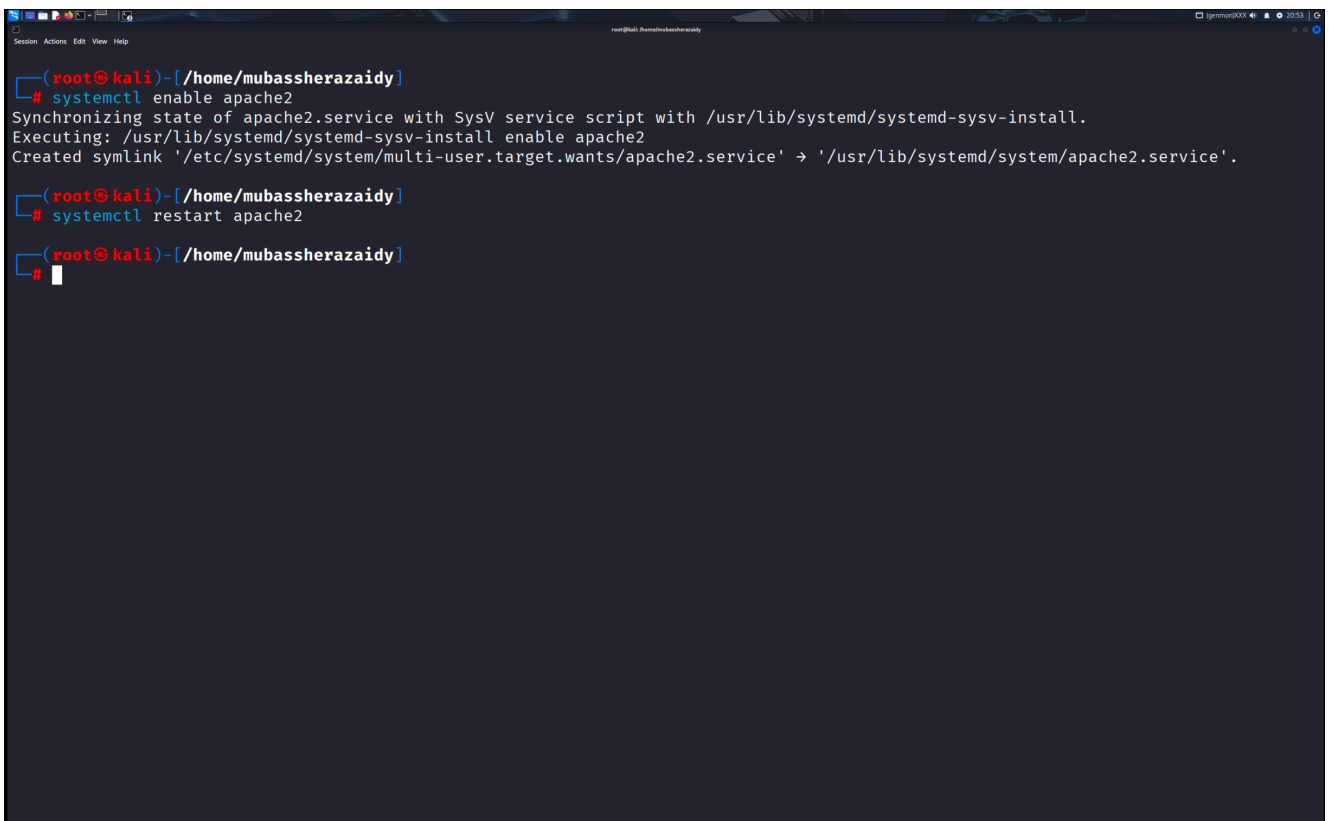
STEP 8 : Run the following command to enable apache :
`systemctl enable apache2`



```
(root@kali)-[/home/mubassherazaidy]
# systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' -> '/usr/lib/systemd/system/apache2.service'.

(root@kali)-[/home/mubassherazaidy]
#
```

STEP 9 : Use `sudo systemctl restart apache2` to reload Apache after configuration changes. Restarting stops the old process and starts a new one, applying the updated SSL settings. Without restarting, Apache would still run with the old configuration.”

A terminal window with a dark background and light blue text. The prompt is `(root@kali)-[/home/mubassherazaidy]`. The first command is `# systemctl enable apache2`, followed by system messages: `Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.`, `Executing: /usr/lib/systemd/systemd-sysv-install enable apache2`, and `Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' -> '/usr/lib/systemd/system/apache2.service'.`. The second command is `# systemctl restart apache2`, followed by a new prompt `(root@kali)-[/home/mubassherazaidy]` and a red prompt character `#` with a cursor.

```
(root@kali)-[/home/mubassherazaidy]
# systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' -> '/usr/lib/systemd/system/apache2.service'.

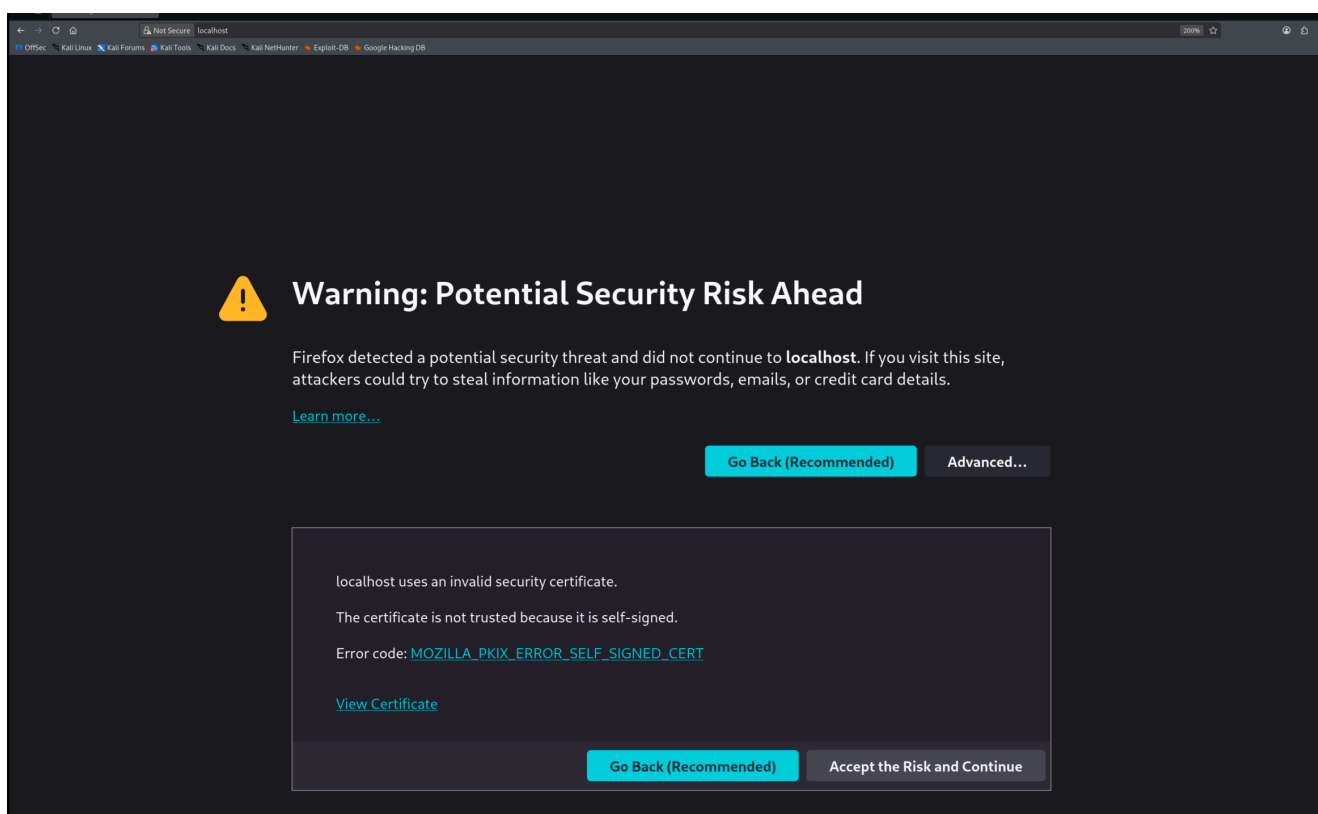
(root@kali)-[/home/mubassherazaidy]
# systemctl restart apache2

(root@kali)-[/home/mubassherazaidy]
#
```

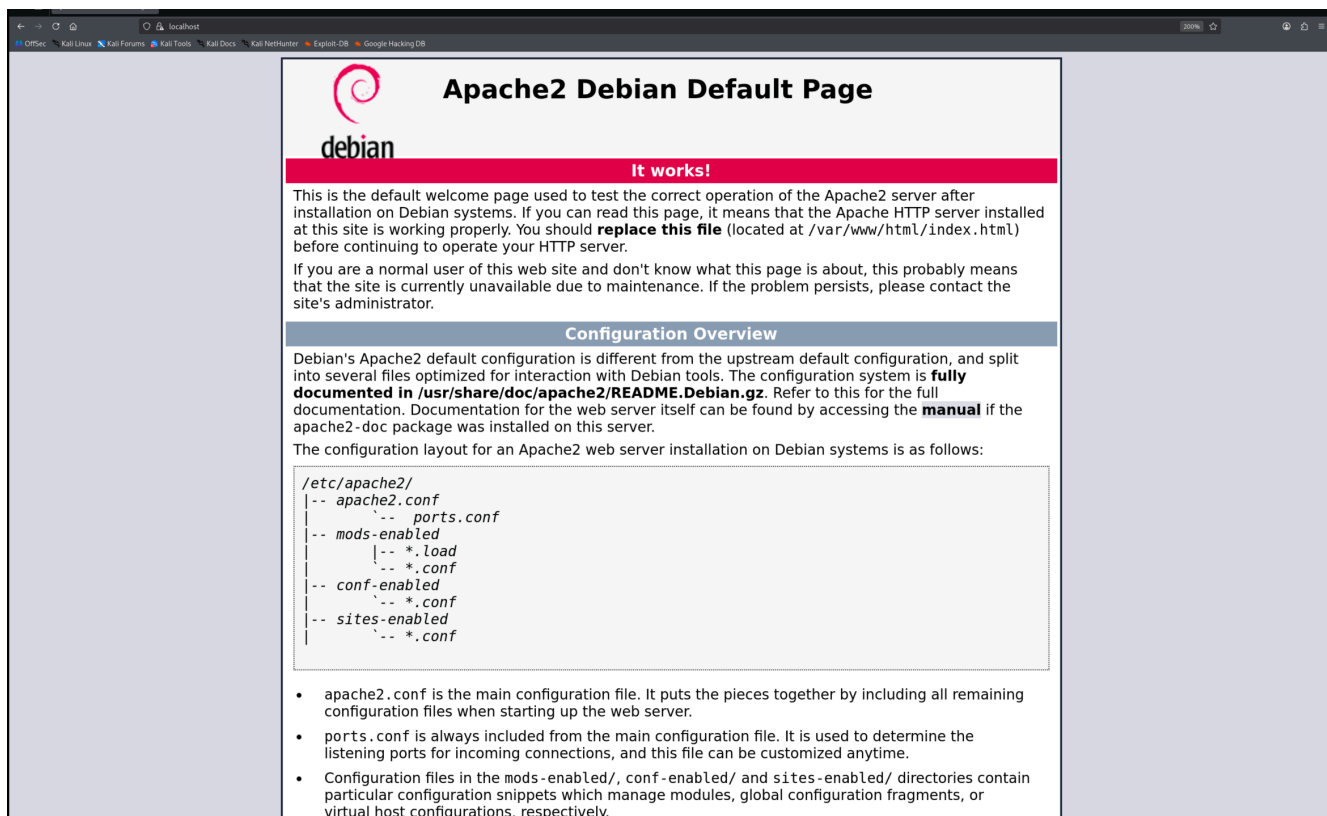
STEP 10 : Test in browser: https://localhost



- The browser warns because the certificate is self-signed, but encryption is active so click on “Accept the Risk and Continue”



- After configurations you should be able to access the Apache server securely using `https://localhost`. The Apache 2 default page confirms that HTTPS is active and the server is functioning correctly.”



The screenshot shows a web browser window with the address bar set to `localhost`. The page title is "Apache2 Debian Default Page". The Debian logo is visible in the top left. A red banner with the text "It works!" is prominently displayed. Below this, a paragraph explains that the page is the default welcome page used to test the correct operation of the Apache2 server after installation on Debian systems. It states that if the page is visible, the Apache HTTP server is working properly and instructs the user to replace the file located at `/var/www/html/index.html` before continuing to operate the HTTP server. Another paragraph explains that if a normal user of the website doesn't know what this page is about, it probably means the site is currently unavailable due to maintenance, and advises contacting the site's administrator. A section titled "Configuration Overview" follows, explaining that Debian's Apache2 default configuration is different from the upstream default and is split into several files optimized for interaction with Debian tools. It mentions that the configuration system is fully documented in `/usr/share/doc/apache2/README.Debian.gz` and refers to the `manual` if the `apache2-doc` package was installed. Below this, it states that the configuration layout for an Apache2 web server installation on Debian systems is as follows: a code block shows the directory structure of `/etc/apache2/`, listing `apache2.conf`, `ports.conf`, `mods-enabled` (containing `*.load` and `*.conf`), `conf-enabled` (containing `*.conf`), and `sites-enabled` (containing `*.conf`). Finally, a bulleted list explains the purpose of these files: `apache2.conf` is the main configuration file; `ports.conf` is used to determine the listening ports; and configuration files in the `mods-enabled`, `conf-enabled`, and `sites-enabled` directories contain particular configuration snippets for modules, global configuration fragments, or virtual host configurations.

Apache2 Debian Default Page

debian

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Debian systems. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Debian's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Debian tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Debian systems is as follows:

```
/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|   |-- mods-enabled
|       |-- *.load
|       |-- *.conf
|   |-- conf-enabled
|       |-- *.conf
|   |-- sites-enabled
|       |-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.
- Configuration files in the `mods-enabled/`, `conf-enabled/` and `sites-enabled/` directories contain particular configuration snippets which manage modules, global configuration fragments, or virtual host configurations, respectively.